

Homework 6

Data Cleansing e Record Linkage di Dati Automotive da Sorgenti Web

Giovanni Pio Grieco
Irene Proietti Cesarini
Silvia Mucci

A.A. 2025/2026

Sommario

Il presente lavoro descrive la progettazione e l'implementazione di una pipeline di **Data Integration** volta all'unificazione di due dataset eterogenei nel settore automobilistico: *Craigslist Cars & Trucks* e *US Used Cars*. L'obiettivo centrale risiede nella risoluzione delle entità (**Entity Resolution**), ovvero nell'identificare se record provenienti da sorgenti distinte si riferiscano alla medesima automobile. La metodologia adottata prevede una fase di pre-processing per l'allineamento degli schemi, la generazione di un *ground-truth* basato sul codice **VIN** e il confronto prestazionale tra diverse strategie di Blocking e tecnologie di Record Linkage, spaziando da approcci basati su regole a modelli avanzati di Deep Learning. I risultati evidenziano il trade-off tra efficacia predittiva e costi computazionali nelle diverse architetture testate.

1 Introduzione

L'integrazione di dati provenienti da sorgenti Web eterogenee rappresenta una delle sfide fondamentali dell'Ingegneria dei Dati, specialmente in domini caratterizzati da elevata frammentazione e rumore come quello del mercato automobilistico dell'usato.

Il presente progetto si pone l'obiettivo di integrare due vasti dataset distribuiti tramite la piattaforma Kaggle:

- **Craigslist Cars & Trucks Dataset** (`vehicles`): una raccolta di annunci pubblicati sulla nota piattaforma di micro-annunci;
- **US Used Cars Dataset** (`used_cars_data`): un dataset esteso contenente inserzioni dettagliate di veicoli usati negli Stati Uniti.

Il problema cardine affrontato è quello della **Entity Resolution**: determinare se due record, pur presentando descrizioni testuali o strutture di attributi differenti, facciano riferimento alla stessa entità fisica nel mondo reale. Tale task di Record Linkage risulta particolarmente complesso a causa della natura non strutturata dei dati sorgente e della possibile presenza di duplicati interni o errori di inserimento.

Per risolvere tale ambiguità, il progetto esplora un'intera pipeline di elaborazione: dall'analisi della qualità dei dati alla valutazione di modelli allo stato dell'arte per il matching, analizzando criticamente l'impatto delle strategie di ottimizzazione sulle prestazioni finali del sistema.

Il codice completo del progetto è disponibile al seguente repository GitHub:

<https://github.com/giovanni-grieco/idd.git>

2 Analisi delle Sorgenti

Le sorgenti analizzate consistono in dataset tabellari di ampie dimensioni contenenti annunci di vendita di autovetture. La natura eterogenea dei dati e la differenza di scala tra i due corpus rappresentano la prima sfida del progetto:

- **Craigslist Cars & Trucks Dataset** comprende circa **426 mila** record;
- **US Used Cars Dataset** presenta una volumetria di circa **3 milioni** di record.

2.1 Qualità dei Dati

Le tabelle presentano differenze significative nel numero e nella denominazione delle colonne, sebbene vi sia una forte corrispondenza semantica "uno-a-uno" tra molti attributi. Esempi rilevanti sono le coppie `model_name/model` e `fuel/fuel_type`.

La qualità dei dati è influenzata dalla presenza diffusa di valori nulli, che in alcuni casi rendono i record del tutto inutilizzabili per i fini dell'integrazione. Inoltre, è stata riscontrata la presenza di duplicati all'interno delle singole sorgenti, rendendo necessaria una fase preliminare di de-duplicazione intra-source oltre al task principale di record linkage inter-source.

ATTRIBUTO	% NULL	% UNICI
bed	99.35	0.01
bed_height	85.7	0.0
bed_length	85.7	0.15
cabin	97.88	0.01
combine_fuel_economy	100.0	0.0
is_certified	100.0	0.0
is_cpo	93.9	0.0
is_oemcpo	95.49	0.0
vehicle_damage_category	100.0	0.0

Figura 1: Attributi con percentuale maggiore del 70% di valori nulli in US Used cars dataset

ATTRIBUTO	% NULL	% UNICI
size	71.77	0.0
county	100.0	0.0

Figura 2: Attributi con percentuale maggiore del 70% di valori nulli in Craigslist dataset

2.2 VIN (Vehicle Identification Number)

Entrambi i dataset condividono l'attributo **VIN**, ovvero il numero di telaio univoco dell'automobile. Nonostante il dataset sia intrinsecamente rumoroso e il VIN possa contenere errori di inserimento, ai fini del progetto si assume tale valore come corretto.

L'identificatore VIN svolge una funzione duplice:

1. **De-duplicazione e Ground Truth:** permette di identificare univocamente le entità per pulire le singole sorgenti e costruire il *ground-truth* necessario per l'addestramento e la valutazione dei modelli;
2. **Validazione:** funge da "oracolo" per misurare le prestazioni del sistema.

È fondamentale sottolineare che il VIN viene **escluso durante** la fase di esecuzione degli algoritmi di **matching**. L'obiettivo del lavoro è infatti quello di validare l'efficacia del record linkage basato esclusivamente sugli attributi descrittivi (come marca, modello e anno), simulando uno scenario reale in cui l'identificatore univoco non sia disponibile.

3 Architettura della Pipeline

Il sistema di integrazione è stato progettato come una pipeline modulare articolata in **quattro fasi** principali: pre-elaborazione, partizionamento dei dati, ottimizzazione tramite blocking e risoluzione delle entità.

3.1 Pre-processing e Allineamento

La fase iniziale è dedicata alla pulizia e alla normalizzazione delle sorgenti analizzate singolarmente:

- **Filtro e De-duplicazione:** i record caratterizzati da un'elevata densità di valori nulli vengono rimossi in quanto non informativi. Contestualmente, si procede alla rimozione dei duplicati intra-sorgente per garantire l'integrità dei dati di partenza;
- **Schema Mapping:** i dataset vengono ricondotti a uno schema mediato comune. Questa operazione prevede la ridenominazione e la selezione degli attributi affinché le due sorgenti risultino strutturalmente speculari. In questa fase, a ogni record viene assegnato un identificatore univoco denominato `row_id`.

Attributo Mediato	Craigslist	Us Used Cars
year	year	year
transmission	transmission	transmission
description	description	description

Figura 3: Schema mediato tra i due dataset sugli attributi omonimi

Attributo Mediato	Craigslist	Us Used Cars
vin	VIN	vin
manufacturer	manufacturer	make_name
model	model	model_name
mileage	odometer	mileage
fuel_type	fuel	fuel_type
body_type	type	body_type
color	paint_color	exterior_color
cylinders	cylinders	engine_cylinders
wheel_system	drive	wheel_system
condition	condition	is_new
lat	lat	latitude
long	long	longitude

Figura 4: Schema mediato tra i due dataset sugli attributi non omonimi

3.2 Generazione della Ground Truth e Campionamento

Sfruttando l'attributo VIN come chiave di volta, è stata generata una **tabella di corrispondenze** (Ground Truth) contenente le coppie di **row_id** che identificano lo stesso veicolo nelle due sorgenti.

Per l'addestramento e la valutazione dei modelli, è stato costruito un dataset bilanciato composto da:

- **4.000 coppie in match** estratte tramite l'identificazione dei VIN comuni;
- **4.000 coppie non in match** selezionate casualmente dalle due sorgenti.

Il **dataset finale** risulta quindi composto da **8.000 coppie etichettate**, garantendo l'assenza di bias statistici durante la fase di apprendimento.

3.3 Split dei dati

Poiché due delle tre tecnologie di *Record Linkage* adottate si basano su paradigmi di Machine Learning, si è reso necessario suddividere il dataset in tre sotto-insiemi distinti:

- **Training Set (70%)**: per l'addestramento dei modelli;
- **Validation Set (15%)**: per l'ottimizzazione degli iperparametri;
- **Test Set (15%)**: per la valutazione finale delle prestazioni.

3.4 Blocking

Prima di procedere al matching vero e proprio, il dataset attraversa uno stadio di Blocking.

Gli algoritmi implementati (**B1** e **B2**) hanno il compito di filtrare le coppie candidate, riducendo drasticamente il numero di confronti necessari. Questa fase è cruciale per garantire la scalabilità del sistema, limitando l'onere computazionale per i classificatori in cascata.

3.5 Entity Matching

L'ultima fase prevede l'elaborazione delle coppie superstiti tramite tre diversi sistemi di Entity Matching. Ogni tecnologia richiede un adattamento specifico del formato dei dati (**data shaping**), gestito tramite moduli di interfaccia dedicati.

L'output finale della pipeline consiste nella classificazione delle coppie come match o non-match, permettendo così la valutazione comparativa dei diversi approcci.

4 Strategie di Blocking

Il Blocking è una fase fondamentale nei processi di *Entity Matching* poiché permette di ridurre drasticamente il numero di confronti necessari, limitando l'analisi alle sole coppie di record che presentano un'alta probabilità di essere dei match.

Sebbene nel contesto del presente progetto la cardinalità ridotta del dataset bilanciato (8.000 righe) non renderebbe il blocking strettamente necessario per ragioni di efficienza, esso viene implementato e analizzato per la valutazione sperimentale dell'intera pipeline.

Il modulo riceve in ingresso la tabella delle coppie candidate e, applicando una delle due strategie definite (B1 o B2), produce in output due sotto-insiemi: le coppie "superstiti" (candidate al matching) e le coppie "escluse" dal processo. Tale distinzione è essenziale per il calcolo delle metriche di valutazione delle performance della pipeline.

4.1 B1: Phonetic Hashing

La strategia **B1** adotta un approccio basato sulla combinazione di attributi chiave per la generazione dei blocchi:

- **Attributi coinvolti:** vengono utilizzati i campi `manufacturer` e `model`;
- **Logica di hashing:** il sistema estrae la radice fonetica degli attributi, li concatena in un'unica stringa e ne calcola l'**hash MD5**;
- **Criterio di inclusione:** se entrambi i record di una coppia producono lo stesso valore hash, vengono assegnati allo stesso blocco e considerati idonei per le fasi successive; in caso contrario, la coppia viene filtrata ed esclusa.

4.2 B2: Multi-key Hashing in OR Logico

La strategia B2 estende la precedente introducendo l'attributo `year` e modificando radicalmente la logica di assegnazione ai blocchi:

- **Generazione di chiavi multiple:** a differenza di B1, per ogni coppia vengono generate quattro diverse chiavi di blocco:
 1. **Hashing fonetico** di `manufacturer` e `model`;
 2. **Hashing** del `manufacturer` combinato con la prima lettera del `model`;
 3. Utilizzo diretto della stringa `model` senza hashing;
 4. Utilizzo dell'attributo `year`.
- **Criterio di inclusione (OR Logico):** una coppia è considerata valida se i due record coincidono in almeno uno dei quattro blocchi generati. Questo approccio "multi-chiave" mira ad aumentare la recall del sistema, riducendo la probabilità di escludere erroneamente match reali a causa di variazioni minime nei dati.

5 Tecnologie di Entity Matching

Per affrontare il task di risoluzione delle entità, sono state implementate e confrontate **tre diverse tecnologie** che rappresentano approcci metodologici distinti: deterministico basato su regole, probabilistico basato su Machine Learning e neurale basato su Deep Learning.

5.1 Python RecordLinkage – Approccio basato su Regole

La libreria **RecordLinkage** è stata utilizzata per implementare un sistema di matching basato su euristiche e soglie deterministiche. In questa configurazione, il sistema valuta la somiglianza tra i record attraverso i seguenti criteri:

- **Attributi testuali:** per il confronto delle stringhe è stata adottata la metrica di similarità di **Jaro-Winkler**, impostando una soglia critica di 0,6;
- **Attributi numerici:** per campi quali **mileage** e **year**, la similarità è stata calcolata applicando una funzione basata sulla **distribuzione gaussiana** (campana di Gauss), permettendo una valutazione più flessibile degli scostamenti numerici.

5.2 Dedupe – Machine Learning e Active Learning

Dedupe è una libreria Python che utilizza modelli di apprendimento automatico per inferire le regole di matching direttamente dai dati. Il processo si articola in due momenti distinti:

- **Fase di Addestramento:** il modello apprende le configurazioni di matching ottimali analizzando il *training set*. Sebbene la libreria supporti l'**Active Learning** (interazione uomo-macchina per l'etichettatura), in questo progetto le risposte alle interrogazioni del sistema sono state fornite programmaticamente sfruttando il ground-truth precostituito;
- **Fase di Inferenza:** le impostazioni apprese vengono salvate e applicate al *test set* per identificare i match.

È importante notare che, sebbene Dedupe includa algoritmi integrati di blocking, tale funzionalità non è stata utilizzata in questa fase poiché i dati forniti in input erano già stati pre-filtrati dai moduli di blocking descritti nel capitolo precedente.

5.3 Ditto – Deep Learning e Language Models

Ditto rappresenta l'approccio allo stato dell'arte, basandosi su modelli linguistici pre-addestrati (**Transformer**) per il task di *entity matching*.

Il sistema opera trattando i record come sequenze testuali, sfruttando la capacità dei modelli linguistici di cogliere relazioni semantiche profonde tra gli attributi. La pipeline prevede l'utilizzo congiunto di *training set* e *validation set* per il fine-tuning del modello, mentre le prestazioni finali vengono misurate esclusivamente sul *test set*.

6 Analisi del Sistema

6.1 Analisi delle Strategie di Blocking

Le due strategie di blocking presentano comportamenti divergenti in termini di efficacia e impatto sulla pipeline:

- **Blocking B1:** adotta una soluzione semplificata basata sull'hashing di produttore e modello. Nonostante l'uso della radice fonetica, la strategia risulta eccessivamente restrittiva, generando un elevato numero di falsi negativi che impatta negativamente sull'F1-Score finale;
- **Blocking B2:** utilizza chiavi multiple in configurazione logica OR. Questa architettura garantisce una recall molto elevata senza aumentare significativamente i falsi positivi, evitando così un incremento eccessivo dell'onere computazionale rispetto alla strategia B1.

6.2 Analisi delle Tecnologie di Matching

L'analisi comparativa evidenzia diversi trade-off tra automazione, precisione e risorse richieste:

- **RecordLinkage**: si è dimostrata un'ottima soluzione basata esclusivamente su CPU. Tuttavia, il limite principale risiede nella necessità di un intervento umano esperto per definire manualmente le soglie di similarità e i metodi di confronto ottimali per ogni colonna;
- **Dedupe**: grazie all'approccio basato su machine learning, risulta più robusto e richiede meno parametri manuali, ma è decisamente più oneroso in termini computazionali. Per evitare errori di memoria (*Out-Of-Memory*), è stato necessario escludere l'attributo "description" dalle fasi di addestramento e inferenza;
- **Ditto**: questo sistema garantisce elevata robustezza ma impone requisiti hardware stringenti. L'esecuzione su CPU risulta proibitiva in termini di tempo, rendendo l'uso della GPU essenziale. L'ottimizzazione richiede una gestione accurata della VRAM tramite la regolazione della *batch-size* e della *max-len* (fissata a 64 caratteri per troncare le descrizioni troppo lunghe).

7 Specifiche Hardware

Tutte le valutazioni e i test prestazionali sono stati condotti sul seguente nodo di calcolo:

- **CPU**: Intel i5-13500
- **GPU**: Nvidia RTX 4070 (12GB VRAM)
- **RAM**: 32GB DDR4 3200MHz
- **Storage**: Samsung SSD 870 QVO 1TB

8 Valutazione del Sistema

8.1 Performance del Blocking

L'analisi dei due algoritmi evidenzia una netta differenza qualitativa:

- **B1 (Naive)**: Presenta un numero elevato di falsi negativi (**1081** nell'analisi totale), penalizzando significativamente la recall (**0.75**) delle pipeline che lo adottano;
- **B2 (Avanzato)**: Dimostra un'efficacia superiore con una recall di **0.96**, riducendo i falsi negativi a **187**. Tale precisione è talmente elevata da rendere quasi superfluo l'intervento di un successivo *entity-matcher*.

Le metriche evidenziate nelle tabelle sottostanti mostrano risultati coerenti sia sull'intero dataset (ALL) che sul *test set* utilizzato per i modelli di Machine Learning.

Blocking	TP	FP	FN	TN	Precision	Recall	F1
B1_ALL	3241	32	1081	4163	0.99	0.75	0.85
B2_ALL	4135	69	187	4126	0.98	0.96	0.97
B1_TEST	1929	15	648	2543	0.99	0.75	0.85
B2_TEST	2459	31	118	2527	0.99	0.95	0.97

Figura 5: Performance registrate dalle strategie di blocking su diversi dataset

8.2 Analisi dei Sistemi di Matching

Il confronto tra i tre sistemi mette in luce il trade-off tra precisione metrica e tempi di elaborazione.

8.2.1 Python RecordLinkage

Valutato su dati pre-filtrati dal blocking, RecordLinkage mantiene una precisione elevata (0.99), ma subisce il limite della recall del blocking quando inserito nella pipeline completa.

	TP	FP	FN	TN	Precision	Recall	F1 Score
RL_B1_ALL	3050	8	191	24	0.99	0.94	0.97
RL_B2_ALL	3816	8	319	61	0.99	0.92	0.96
PIPELINE_RL_B1_ALL	3050	8	1272	4187	0.99	0.71	0.83
PIPELINE_RL_B2_ALL	3816	8	506	4187	0.99	0.88	0.94

Figura 6: Performance registrate dalla libreria RecordLinkage su diversi dataset

8.2.2 Dedupe

Dedupe si rivela il sistema più performante sui dati in blocco, raggiungendo una recall di **1.0** e un F1-Score di **0.99** sul *test set*. La pipeline completa con B2 beneficia di questa precisione, stabilizzandosi su un F1 di **0.97**.

	TP	FP	FN	TN	Precision	Recall	F1 Score
DEDUPE_B1_TEST	1929	15	0	0	0.99	1	0.99
DEDUPE_B2_TEST	2455	28	4	3	0.99	0.99	0.99
PIPELINE_DEDUPE_B1_TEST	1929	15	648	2543	0.99	0.75	0.85
PIPELINE_DEDUPE_B2_TEST	2455	28	122	2530	0.99	0.95	0.97

Figura 7: Performance registrate da Dedupe su diversi dataset

8.2.3 Ditto

Ditto conferma la sua robustezza con un F1-Score di **0.97** sui dati in blocco. Tuttavia, presenta i costi computazionali più elevati: il tempo di addestramento è oltre dieci volte superiore a quello di Dedupe, e l'inferenza è significativamente più lenta.

	TP	FP	FN	TN	Precision	Recall	F1 Score
DITTO_B1_TEST	1817	2	112	13	0.99	0.94	0.97
DITTO_B2_TEST	2314	7	145	24	0.99	0.94	0.97
PIPELINE_DITTO_B1_TEST	1817	2	760	2556	0.99	0.71	0.83
PIPELINE_DITTO_B2_TEST	2314	7	263	2551	0.99	0.90	0.94

Figura 8: Performance registrate da Ditto su diversi dataset

8.2.4 Efficienza Computazionale

Il confronto dei tempi sottolinea il forte impatto hardware richiesto dai modelli basati su Deep Learning.

System	avg Training Time (s)	avg Inference Time (s)
Dedupe	41	1
Ditto	503	81

Figura 9: Tempi di addestramento dei modelli Dedupe e Ditto

9 Conclusioni

L'analisi comparativa condotta in questo progetto permette di trarre conclusioni rilevanti sull'efficacia delle diverse architetture di integrazione dati testate.

La sperimentazione ha confermato che la strategia di **Blocking B2** (multi-key hashing in OR logico) è nettamente superiore alla soluzione B1.

L'efficacia di tutti e tre i sistemi di entity matching è risultata fortemente dipendente dalla qualità del filtraggio operato a monte. B2 ha dimostrato una tale precisione, con un numero estremamente ridotto di falsi positivi, da generare risultati comparabili a quelli di un sistema di matching completo. Per dataset con caratteristiche simili a quelli analizzati, si può concludere che l'applicazione diretta di una strategia di blocking robusta come B2 potrebbe risultare sufficiente, rendendo superfluo l'impiego di matcher complessi in cascata.

Ciascuna tecnologia di *Entity Matching* testata ha evidenziato specifici vantaggi e criticità:

- **RecordLinkage**: Sebbene sia la soluzione meno onerosa in termini di risorse computazionali, presenta lo svantaggio di richiedere un elevato intervento manuale. L'onere della scelta dei metodi di comparazione e delle soglie di similarità ricade interamente sull'operatore, richiedendo una profonda conoscenza del dominio e del dataset;
- **Ditto**: Grazie all'impiego di *Language Models*, si conferma il sistema più robusto nel gestire dati rumorosi o non strutturati. Tuttavia, i requisiti hardware stringenti e la necessità di una GPU performante ne limitano l'applicabilità in scenari privi di infrastrutture dedicate, rendendolo meno efficiente nel bilanciamento tra costi e benefici;
- **Dedupe**: È stato identificato come il sistema più equilibrato e completo per contesti reali. La sua capacità di operare su hardware di media capacità (CPU-based) e la flessibilità offerta dall'**Active Learning** lo rendono ideale per scenari privi di *ground-truth* preesistente. Sebbene nel progetto l'interazione sia stata automatizzata, la possibilità di addestrare il modello tramite feedback interattivi rappresenta un valore aggiunto fondamentale per la scalabilità del processo di integrazione.

In sintesi, la scelta della pipeline ottimale dipende dal trade-off tra la qualità dei dati di partenza e le risorse hardware disponibili. Per l'integrazione di sorgenti automotive eterogenee, l'accoppiata tra una strategia di **Blocking multi-chiave** e un sistema di **Machine Learning adattivo** come Dedupe rappresenta la soluzione più sostenibile e performante.